

Efficient Polynomial Multiplication for HQC on ARM Cortex-M4

Jihoon Jang¹, Myeonghoon Lee¹, Donggeun Kwon², Seokhie Hong³, Suhri Kim⁴ and Sangjin Lee¹

¹ Korea University, Seoul, South Korea, {jhw1031506, ope2527, sangjin}@korea.ac.kr

² Korea Testing & Research Institute, Pangyo, South Korea, donggeun.kwon@gmail.com

³ Contec, Daejeon, South Korea, shhong@contec.kr

⁴ Sungshin Women's University, Seoul, South Korea, suhrikim@sungshin.ac.kr

Abstract.

HQC, a code-based Key Encapsulation Mechanism (KEM) recently selected for NIST post-quantum cryptography standardization, utilizes polynomial multiplication over $\mathbb{F}_2[x]$, the efficiency of which determines the overall performance of the scheme on embedded targets. On the ARM Cortex-M4, where carry-less multiplication instructions are unavailable, prior works have focused on the Frobenius Additive FFT (FAFFT) and radix-16 methods. However, for polynomial lengths $n = 2^k + r$ with small r , such as those in HQC-1 and -3, applying FAFFT requires a transform length of $2^{k+2} \approx 4n$, introducing substantial padding overhead.

In this paper, we propose three optimized approaches to enhance HQC performance on the Cortex-M4: (i) Hybrid FAFFT-CRT, (ii) Hybrid FAFFT-Karatsuba, and (iii) a radix-16 method. To reduce the padding overhead of FAFFT, we use the Hybrid FAFFT-CRT and FAFFT-Karatsuba methods. The Hybrid FAFFT-CRT method maps the polynomial ring to a product of a FAFFT-friendly ring and a small-degree polynomial ring, and the Hybrid FAFFT-Karatsuba method utilizes a 2-way split to align subproducts with smaller power-of-two FAFFT lengths. We further optimize the core FAFFT butterfly operation by shortening XOR sequences and improving register scheduling. For the radix-16 approach, we apply it to HQC for the first time and develop an operation-count cost model to identify optimal Karatsuba and Toom-Cook combinations.

On a NUCLEO-L4R5ZI board with a Cortex-M4 microcontroller, all of these methods are more efficient than the state-of-the-art for HQC-1 and -3. The Hybrid FAFFT-CRT method reduces polynomial multiplication cycles by 33.4% and 27.2% for HQC-1 and -3, respectively. This leads to cycle reductions for key generation, encapsulation, and decapsulation by 23.7%, 24.2%, and 21.9% for HQC-1, and 18.0%, 18.6%, and 17.7% for HQC-3. For HQC-5, our optimized butterfly FAFFT provides a consistent speedup of 1.4 - 1.6% across all KEM operations.

Keywords: Additive FFT, Radix-16 representation, Finite field multiplication, Cortex-M4, HQC

1 Introduction

HQC [AAB⁺22] is a key encapsulation mechanism (KEM) that was officially selected as a NIST standard for post-quantum cryptography in March 2025. Since polynomial multiplication over $\mathbb{F}_2[x]$ is the core operation on HQC, the efficiency of HQC depends on how fast polynomial multiplication over $\mathbb{F}_2[x]$ can be performed. In particular, on embedded targets such as ARM Cortex-M4, which lack carry-less multiplication instructions, polynomial multiplication dominates the overall execution time. To address this issue,

two implementation approaches are especially relevant on such platforms. The first is the Frobenius Additive FFT (FAFFT), which reduces evaluation points by exploiting the Frobenius map. The second is a radix-16 method, which turns $\mathbb{F}_2[x]$ polynomial multiplications into ordinary 32-bit integer multiplications that the Cortex-M4 can execute efficiently.

1.1 Contributions

This work considers both approaches and optimizes them for the polynomial degrees used in HQC. In particular, for FAFFT, our focus is on reducing the overhead that arises from applying the method to polynomial degrees of the form $n = 2^d + r$ with small r , such as in HQC-1 and -3. Since FAFFT operates on lengths of a power of 2, the previous approach requires a transform length of 2^{d+2} , which is nearly $4n$, leading to significant overhead. To address this, we propose two hybrid methods. First, we apply the Hybrid FAFFT-CRT method, which maps the polynomial ring to a product of a FAFFT-friendly ring of size 2^{d+1} and a small-degree polynomial ring, recombining them via the Chinese Remainder Theorem. Second, we apply a Hybrid FAFFT-Karatsuba method using a 2-way split at 2^d . The subproducts of size 2^d are evaluated using a FAFFT of size 2^{d+1} , while the residual of size r is handled via radix-16 multiplication. Additionally, in the FAFFT butterfly stage, we shorten the XOR sequences resulting from fixed-coefficient multiplications and reduce clock cycles by scheduling operations to utilize all 14 general-purpose registers. We also optimize bit permutations between registers using **SWAPMOVE**. For radix-16 multiplication, we use a cost model based on operation counts to explore Karatsuba and Toom-Cook combinations, producing optimal candidates that are validated on the target device.

The main contributions of this paper are summarized as follows:

- We apply the Hybrid FAFFT-CRT method to efficiently handle polynomial degrees that do not align well with FAFFT. By mapping the polynomial ring $\mathbb{F}_2[x]/(x^n - 1)$ to a ring $\mathbb{F}_2[x]/(\mathcal{P}_{d+1}(x) \cdot x^{r_c})$, where $\mathbb{F}_2[x]/\mathcal{P}_{d+1}(x)$ corresponds to the FAFFT of size 2^{d+1} and $\mathbb{F}_2[x]/x^{r_c}$ covers the residual, this method avoids the large padding overhead of the previous method of FAFFT. During the reconstruction process via the Chinese Remainder Theorem (CRT), we optimized the multiplication between $\mathcal{P}_{d+1}(x)$ and intermediate polynomials as a sparse-dense product tailored for the ARM Cortex-M4 architecture.
- We propose the Hybrid FAFFT-Karatsuba method to handle small residual part during polynomial multiplication. To the best of our knowledge, this is the first work to optimize FAFFT using a 2-way Karatsuba split for polynomial degrees of the form $n = 2^d + r$ with small r . Since FAFFT has a complexity of $O(n \log n)$, performing two FAFFT evaluations of size 2^{d+1} plus one small-degree multiplication for r is cheaper than a single FAFFT of size 2^{d+2} when r is sufficiently small.
- We optimize the FAFFT butterfly algorithm for the ARM Cortex-M4. For the multiplications by constant coefficients $v_{17}, \dots, v_{27}$ within the butterfly stages, we shorten the XOR sequences and reduce the cycle count by scheduling operations to fully saturate all 14 available general-purpose registers. Furthermore, in the final five layers, we utilize the **SWAPMOVE** operation to minimize the overhead of required bit swaps between registers.
- We apply radix-16 multiplication to HQC for the first time, to the best of our knowledge, and utilize a cost model based on operation counts to identify optimal Karatsuba and Toom-Cook combinations without relying on a measurement-based method. This avoids inaccuracies from conditional branches and table lookups during the search. Under the same algorithm sets, we compare against the **gf2x** library for

the three HQC degrees. The resulting combination is identical for HQC-1 and HQC-3, while for HQC-5 our method produces a more efficient combination.

- We compare our implementations with the state-of-the-art Cortex-M4 implementation [LJKH25]. For HQC-1 and HQC-3, our Hybrid FAFFT-CRT method achieves the best performance. Specifically, for HQC-1, it reduces clock cycles by 23.7%, 24.2%, and 21.9% for key generation, encapsulation, and decapsulation, respectively. For HQC-3, it reduces them by 18.0%, 18.6%, and 17.7%. For HQC-5, although both hybrid FAFFT is less effective due to the large residual and the polynomial degree is too high for radix-16 method, our butterfly-optimized FAFFT achieves a performance gain of 1.4 - 1.6% over the previous state-of-the-art across all KEM operations.

The source code and implementation details of the proposed method are available at: <https://github.com/jhw1031506/Efficient-Polynomial-Multiplication-for-HQC-on-ARM-Cortex-M4>.

1.2 Related Work

The ARM Cortex-M4 serves as the baseline embedded evaluation platform in the NIST PQC standardization project. As noted in NIST IR 8413, performance of PQC algorithms on embedded devices is one of the evaluation criteria. Specifically, the Cortex-M4 has been widely adopted in academic literature as a representative embedded platform because of its balance between resource constraints and computational capabilities. NIST has also formally acknowledged its prevalence and recommended it as an optimization target for the second round of the PQC project. As a result, many PQC schemes have been optimized for Cortex-M4 [BKS19, ABCG20, CHK⁺21, CC21, CCK21, GKS21, ACC⁺22, AHKS22, HZZ⁺22, HAZ⁺24, LJKH25]. In particular, pqm4 serves as the standard testing and benchmarking infrastructure for Cortex-M4 implementations [KPR⁺]. Code-based schemes are no exception. Because the performance of code-based schemes such as HQC, BIKE, and Classic McEliece is largely determined by the efficiency of polynomial multiplication, prior research has focused on two main optimization approaches.

The first approach is to use an additive FFT. Cantor [Can89] described how to compute length $n = 2^m$ FFTs over $\mathbb{F}_{2^m}$ efficiently when m is a power of two, with further improvements in [VzGG96, GM10]. Especially, in [GM10], additive FFT is optimized by introducing new algorithms based on Taylor expansion over finite fields of characteristic two, which significantly reduce the number of multiplications compared to previous approaches in [VzGG96]. Based on this, Li et al. [LCK⁺18] introduced the Frobenius Additive FFT (FAFFT). By using the Frobenius map, FAFFT reduces the number of evaluation points required for FFTs of length 2^m , leading to significantly faster binary polynomial multiplication compared to previous additive FFT techniques. Using FAFFT, Chen et al. [CCK21] and Lee et al. [LJKH25] optimized BIKE and HQC on Cortex-M4, respectively. Furthermore, to address the inefficiency of additive FFT when polynomial degrees are slightly larger than a power of two, Chen et al. [CCPY26] proposed a novel combination of additive FFT with the CRT. While their implementation targeted high-performance platforms such as x86-64 and ARM NEON, they demonstrated that the CRT-based approach allows FFT-based implementations to outperform traditional Toom-Cook and Karatsuba methods for HQC parameters.

The second approach is to use a radix-16 representation. This approach was proposed in [CC21], and the paper demonstrates an optimized implementation of Classic McEliece on Cortex-M4. In particular, the paper introduces a radix-16 representation to efficiently perform field multiplications required in the Berlekamp–Massey algorithm. By exploiting the structure of radix-16 polynomials and using the `umlal` instruction to emulate carryless multiplications, the implementation achieves low-latency multiplication and inversion

operations in $\mathbb{F}_{2^{12}}$ and $\mathbb{F}_{2^{13}}$, resulting in significant speedups on 32-bit microcontrollers. Chen et al. [CGKT22] apply the approach to BIKE, using Karatsuba and Toom-Cook for large-degree polynomial multiplication and selecting the optimal combination with the `gf2x` library.

1.3 Organization

2 Frobenius Additive FFT

2.1 Additive FFT using novel polynomial basis

In 2014, Lin et al. proposed a method for performing the Additive FFT (AFFT) using a novel polynomial basis [LCH14]. This basis is constructed as a product of vanishing polynomials defined over the Cantor basis.

2.1.1 Cantor basis and vanishing polynomial

The Cantor basis, introduced by Cantor in 1989 [Can89], is a specially constructed basis used for efficient computation of the Fast Fourier Transform (FFT) over binary extension fields $\mathbb{F}_{2^m}$, where m is a power of two. Cantor basis enables AFFT algorithms by leveraging the structure of $\mathbb{F}_{2^m}$ as a vector space over $\mathbb{F}_2$.

Let the Cantor basis be defined as the ordered set $\{v_0, v_1, \dots, v_{m-1}\} \subset \mathbb{F}_{2^m}$, where the elements satisfy the following recurrence relation:

$$v_0 = 1, \quad v_{i+1}^2 + v_{i+1} = v_i, \quad \text{for } 0 \leq i < m-1.$$

Using this basis, we define a sequence of subspaces $W_k \subseteq \mathbb{F}_{2^m}$ as the linear span over $\mathbb{F}_2$ of the first k basis elements:

$$W_k = \text{span}_{\mathbb{F}_2}\{v_0, v_1, \dots, v_{k-1}\}, \quad \text{with } W_0 = \{0\}.$$

To represent the elements of W_k , we adopt a binary expansion encoding. For each $k \geq 1$, the $(i+1)$ -th element $\omega_i \in W_k$ is expressed as a linear combination:

$$\omega_i = i_0 v_0 + i_1 v_1 + \dots + i_{k-1} v_{k-1},$$

where the coefficients $i_j \in \{0, 1\}$ are obtained from the binary expansion

$$i = i_0 + 2i_1 + 2^2 i_2 + \dots + 2^{k-1} i_{k-1}.$$

We further define a family of polynomials $\{s_i(x)\}$ recursively as follows:

$$s_0(x) = x, \quad s_{i+1}(x) = (s_i(x))^2 + s_i(x), \quad \text{for } i \geq 0.$$

The basis elements v_i , the subspaces W_i , and the polynomials $s_i(x)$, for $0 \leq i < m$, satisfy the following properties:

- $\deg(s_i(x)) = |W_i| = 2^i$.
- The polynomials satisfy the identity $s_i(x+y) = s_i(x) + s_i(y)$, and $s_k(x) = x^{2^k} + x$ if and only if k is a power of two.
- $s_i(x+y) = s_i(x) + s_i(y)$, $s_{i+j}(x) = s_i(s_j(x))$.
- The polynomial $s_k(x)$ can be represented as a product over the subspace W_k :

$$s_k(x) = \prod_{a \in W_k} (x - a),$$

and is referred to as a *vanishing polynomial* over W_k .

- The evaluation of $s_k(x)$ at basis elements v_i satisfies:

$$s_k(v_i) = \begin{cases} v_{i-k}, & \text{if } i > k. \\ 1, & \text{if } i = k. \\ 0, & \text{if } i < k. \end{cases}$$

2.1.2 Novel polynomial basis and Additive FFT

In 2014, Lin et al. introduced a novel polynomial basis $\{X_k(x)\}$ for efficient AFFT computations. Each basis polynomial $X_k(x)$ is defined as a product of vanishing polynomials:

$$X_k(x) = \prod_i (s_i(x))^{b_i},$$

where the index k is represented in binary as $k = \sum b_i 2^i$, with $b_i \in \{0, 1\}$. Since $b_i = 1$ if and only if the i -th bit of k is 1, the degree of $X_k(x)$ is computed as

$$\deg(X_k(x)) = \sum b_i \cdot 2^i = k.$$

Thus, the basis polynomial $X_k(x)$ has degree exactly k , matching the binary expansion of its index. Based on this property, any polynomial $f(x)$ over $\mathbb{F}_{2^m}$ of degree less than n can be uniquely represented in the form:

$$f(x) = g_0 + g_1 X_1(x) + \cdots + g_{n-1} X_{n-1}(x),$$

where each $g_i \in \mathbb{F}_{2^m}$.

Novel polynomial basis $\{X_k(x)\}$ satisfies the following properties:

- $X_{a \cdot 2^i}(x) \cdot X_b(x) = X_{a \cdot 2^i + b}(x)$ if $b < 2^i$.
- $X_i(s_j(x)) = X_{i \cdot 2^j}(x)$

In a subsequent work [LANH16], the authors proposed an efficient basis conversion algorithm that transforms a polynomial expressed in the monomial basis into its representation in the novel polynomial basis. The algorithm exploits the recursive structure of the vanishing polynomials $s_i(x)$, particularly when $i = 2^k$, where $s_i(x) = x^{2^i} + x$.

The conversion process follows these steps:

1. Identify the largest $i = 2^k$ such that $\deg(s_i(x)) < n$.
2. Express $f(x)$ as a polynomial in $s_i(x) = x^{2^i} + x$:

$$f(x) = h(x, s_i(x)) = \hat{h}(s_i(x)) = h_0(x) + h_1(x) \cdot s_i(x) + \cdots + h_t(x) \cdot s_i^t(x),$$

where $h_j(x) = \sum_{r=0}^{2^i-1} h_{j,r} x^r$ are polynomials with $\deg(h_j(x)) < \deg(s_i(x)) = 2^i$. Let $y = s_i(x)$, then $f(x)$ can be represented as follows:

$$f(x) = \sum_{r=0}^{2^i-1} \{h_{0,r} + h_{1,r} y + \cdots + h_{t,r} y^t\} \cdot x^r.$$

3. Recursively apply the conversion to $h_{0,r} + h_{1,r}y + \dots + h_{t,r}y^t$. Since $X_{t'}(y) = X_{t'}(s_i(x)) = X_{2^i \cdot t'}(x)$, $f(x)$ can be represented as follows:

$$\begin{aligned}
 f(x) &= \sum_{r=0}^{2^i-1} \{q_{0,r} + q_{1,r}X_1(y) + \dots + q_{t,r}X_t(y)\} \cdot x^r. \\
 &= \sum_{r=0}^{2^i-1} \{q_{0,r} + q_{1,r}X_{2^i}(x) + \dots + q_{t,r}X_{2^i \cdot t}(x)\} \cdot x^r. \\
 &= \sum_{t'=0}^t \{q_{t',0} + q_{t',1}x + \dots + q_{t',2^i-1}x^{2^i-1}\} X_{2^i \cdot t'}(x) \\
 &= \sum_{t'=0}^t q_{t'}(x) \cdot X_{2^i \cdot t'}(x)
 \end{aligned}$$

4. Recursively apply the conversion to $q_{t'}(x)$. Since $X_{2^i \cdot t'}(x) \cdot X_j(x) = X_{2^i \cdot t' + j}(x)$, we can write $f(x)$ in the novel polynomial basis as follows:

$$\begin{aligned}
 f(x) = g(X) &= \sum_{t'=0}^t \{g_{2^i \cdot t'} + g_{2^i \cdot t' + 1}X_1(x) + \dots + g_{2^i \cdot t' + 2^i - 1}X_{2^i - 1}(x)\} \cdot X_{2^i \cdot t'}(x) \\
 &= g_0 + g_1X_1(x) + \dots + g_{n-1}X_{n-1}(x).
 \end{aligned}$$

The full procedure is summarized in Algorithm 1, denoted as BASISCVT. Since all operations are performed over $\mathbb{F}_2$, the algorithm consists entirely of bitwise XORs.

Algorithm 1 BASISCVT: Monomial to Novel Polynomial Basis Conversion

Require: Monomial basis polynomial $f(x) = f_0 + f_1x + \dots + f_{n-1}x^{n-1}$

Ensure: $g(X) = g_0 + g_1X_1(x) + \dots + g_{n-1}X_{n-1}(x)$ in novel polynomial basis

- 1: **if** $\deg(f(x)) \leq 1$ **then**
 - 2: **return** $f_0 + f_1X_1$
 - 3: **end if**
 - 4: Find the largest $i = 2^k$ such that $\deg(s_i(x)) \leq \deg(f(x))$
 - 5: Compute $h(x, y) = \hat{h}(y) = h_0(x) + h_1(x)y + \dots + h_t y^t$ s.t. $f(x) = h(x, s_i(x))$. Here $\deg(h_j(x)) < 2^i$.
 - 6: Compute $h'(Y) = q_0(x) + \dots + q_t(x)X_{2^i \cdot t}(x) \leftarrow \text{BASISCVT}(\hat{h}(y))$.
 - 7: Compute $g_{2^i \cdot t'} + g_{2^i \cdot t' + 1}X_1(x) + \dots + g_{2^i \cdot t' + 2^i - 1}X_{2^i - 1}(x) \leftarrow \text{BASISCVT}(q_{t'}(x))$ for all coefficient $q_{t'}(x)$ of $h'(Y)$.
 - 8: **return** $g(X) = g_0 + g_1X_1(x) + \dots + g_{n-1}X_{n-1}(x)$
-

Once the input polynomial $f(x)$ has been converted into the novel polynomial basis form $g(X)$ using the BASISCVT procedure, the next step is to evaluate the polynomial at multiple points efficiently as part of AFFT. Specifically, the butterfly operation is used to compute f at $n = 2^l$ points of the form $\alpha + \omega$, where $\omega \in W_l$ and $\alpha \in \mathbb{F}_{2^m}$.

This step is performed by the BTFY algorithm, which recursively divides the polynomial $g(X)$ into two parts, $p_0(X)$ and $p_1(X)$, and computes the intermediate values $h_0(X)$ and $h_1(X)$ as follows:

$$\begin{aligned}
 h_0(X) &\leftarrow p_0(X) + s_i(\alpha) \cdot p_1(X), \\
 h_1(X) &\leftarrow h_0(X) + s_i(v_i) \cdot p_1(X),
 \end{aligned}$$

where $s_i(x)$ is the vanishing polynomial of degree 2^i corresponding to the largest power of two less than the target evaluation length. Since $s_i(v_i) = 1$, each recursive step of the

butterfly algorithm splits the polynomial and performs two field additions and one field multiplication. The full butterfly operation is detailed in Algorithm 2.

Algorithm 2 BTFY: Butterfly Operation for Additive FFT

Require: Polynomial $g(X) = f(x)$ of degree n in novel polynomial basis, and offset $\alpha \in \mathbb{F}_{2^m}$

Ensure: Evaluation list $[f(\omega_0 + \alpha), f(\omega_1 + \alpha), \dots, f(\omega_{2^l-1} + \alpha)]$ where $\omega_j \in W_l$

- 1: **if** $\deg(g(X)) = 0$ **then**
 - 2: **return** $[g_0]$
 - 3: **end if**
 - 4: Find $i \leftarrow \lceil \log_2 n \rceil - 1$ such that $\deg(s_i(x)) = 2^i < n$
 - 5: Split $g(X)$ as $g(X) = p_0(X) + X_{2^i}(X) \cdot p_1(X) = p_0(X) + s_i(x) \cdot p_1(X)$
 - 6: Compute $h_0(X) \leftarrow p_0(X) + s_i(\alpha) \cdot p_1(X)$
 - 7: Compute $h_1(X) \leftarrow h_0(X) + s_i(v_i) \cdot p_1(X)$
 - 8: **return** $[\text{BTFY}(h_0(X), \alpha), \text{BTFY}(h_1(X), v_i + \alpha)]$
-

The BTFY algorithm exploits the recursive structure of the novel polynomial basis and the properties of vanishing polynomials to perform Additive FFT efficiently. Since each recursive step handles half the size of the input and involves minimal operations, it achieves efficient evaluation over W_l .

Consequently, the complete process for performing the additive FFT on a polynomial $f(x) \in \mathbb{F}_{2^m}[x]$ of degree less than n consists of two main stages. First, the polynomial is transformed from the monomial basis into the novel polynomial basis using the BASISCVT algorithm, as described in the previous section.

With the polynomial represented as $g(X)$ in the novel basis, the second stage employs the BTFY algorithm. This step recursively evaluates $g(X)$ at $n = 2^l$ points of the form $\alpha + \omega$, where $\omega \in W_l$ and $\alpha \in \mathbb{F}_{2^m}$. These recursive butterfly operations enable the efficient evaluation of $f(x)$ over the set $\alpha + W_l$.

This two-step procedure is summarized in Algorithm 3, denoted as AFFT.

Algorithm 3 AFFT: Additive FFT Algorithm

Require: Polynomial $f(x) = f_0 + f_1x + \dots + f_{n-1}x^{n-1} \in \mathbb{F}_{2^m}[x]$

Ensure: Evaluation values $\{f(\omega) : \omega \in W_n\}$

- 1: Convert to novel basis: $g(X) \leftarrow \text{BASISCVT}(f(x))$
 - 2: **return** $\text{BTFY}(g(X), 0)$
-

2.2 Frobenius Additive FFT

2.2.1 Frobenius map and Frobenius partition

The Frobenius Additive FFT (FAFFT) [LCK⁺18, CCK21] employs the Frobenius map to reduce the number of evaluation points required for polynomial multiplication compared with AFFT. Concretely, if AFFT evaluates a polynomial at 2^l points in the coset $\alpha + W_l$, FAFFT evaluates only $2^l/m$ points, where m is the extension degree of $\mathbb{F}_{2^m}$. For example, when $l = 16$ and $m = 32$, FAFFT requires only $2^{16}/32 = 2,048$ evaluations to achieve the same result.

Frobenius map. Let $\phi_2 : \mathbb{F}_{2^m} \rightarrow \mathbb{F}_{2^m}$ denote the *Frobenius map* $\phi_2(a) = a^2$. For any polynomial $f(x) \in \mathbb{F}_2[x] \subset \mathbb{F}_{2^m}[x]$ and any $a \in \mathbb{F}_{2^m}$, it holds that $f(\phi_2(a)) = \phi_2(f(a))$.

Order and partition under Frobenius. Given a finite set $\Sigma \subseteq \mathbb{F}_{2^m}$, the *order* of the Frobenius map ϕ_2 for a set Σ , denoted $\text{Ord}_{\phi_2}(\Sigma)$, is the smallest $j \in \mathbb{N}$ such that $\Sigma = \phi_2^j(\Sigma)$. Σ is called a *Frobenius partition* of Ω if

$$\Omega = \Sigma \cup \phi_2(\Sigma) \cup \dots \cup \phi_2^{j-1}(\Sigma)$$

with pairwise disjoint unions, where $j = \text{Ord}_{\phi_2}(\Sigma)$.

Properties with respect to the Cantor basis. Let $\{v_0, \dots, v_{m-1}\}$ be the Cantor basis of $\mathbb{F}_{2^m}$. Then the following hold:

- $\text{Ord}_{\phi_2}(v_i) = 2 \cdot 2^{\lfloor \log_2 i \rfloor}$ for $i > 0$.
- $\phi_2(W_i) = W_i$ for all i .

Let $k = l - \log_2 m < \frac{m}{2}$ and define $\Sigma := v_{k+m/2} + W_k$. Then, by the properties above, the following properties hold:

- $\text{Ord}_{\phi_2}(\Sigma) = m$ and $|\Sigma| = 2^k = 2^l/m$.
- $\Omega := \Sigma \cup \phi_2(\Sigma) \cup \dots \cup \phi_2^{m-1}(\Sigma)$ has size $|\Omega| = 2^l$.
- $\phi_2^j(\Sigma)$ are disjoint sets for $0 \leq j < m$.

From these properties, it follows that $\Sigma = v_{k+m/2} + W_k$ is a Frobenius partition of Ω . Therefore, evaluating f at the $|\Sigma| = 2^k = 2^l/m$ points recovers the full set of 2^l values by applying ϕ_2 . This allows us to truncate the first $\ell_m = \log_2 m$ butterfly layers in BTFY.

Vanishing polynomial and the ring structure. The set Ω constructed above serves as the evaluation domain for FAFFT. We define the modulus polynomial $\mathcal{P}_l(x)$ as the vanishing polynomial over Ω :

$$\mathcal{P}_l(x) = \prod_{\beta \in \Omega} (x - \beta).$$

Since Ω consists of $2^l = 2^{k+\ell_m}$ distinct elements, $\mathcal{P}_l(x)$ is a polynomial of degree 2^l . Unlike AFFT which typically utilizes a subspace vanishing polynomial $s_l(x)$, FAFFT uses $\mathcal{P}_l(x)$ constructed from the Frobenius orbits of Σ . A key property of FAFFT is that the point-wise multiplication of evaluations over Ω corresponds to polynomial multiplication modulo $\mathcal{P}_l(x)$. Thus, the FAFFT algorithm effectively computes the product in the ring $\mathbb{F}_2[x]/\mathcal{P}_l(x)$.

2.2.2 Multiplication over $\mathbb{F}_2[x]$ with Frobenius Additive FFT

In this section, we describe how to multiply two polynomials $f, g \in \mathbb{F}_2[x]$ using FAFFT. FAFFT proceeds in three stages: BASISCVT, ENCODE, and BTFY. (i) The BASISCVT step (Algorithm 1) converts the input from the monomial basis to the novel polynomial basis. (ii) The ENCODE step uses the Frobenius partition to truncate the first $\ell_m = \log_2 m$ butterfly layers, reducing the set of points from n to n/m . (iii) The remaining layers are performed by the recursive BTFY procedure (Algorithm 2), which evaluates the polynomial at the n/m points. The structure of the algorithm is based on Algorithm 4.

Encode and Decode (truncation of the first ℓ_m butterfly layers). The ENCODE step corresponds to the first $\ell_m = \log_2 m$ butterfly layers of FAFFT. In these layers, at each node of BTFY only one of the two recursive calls is executed, whereas in the remaining $l - \ell_m$ layers both recursive calls are executed as in AFFT. Consequently, FAFFT performs explicit evaluations at only n/m points.

Moreover, this truncation prevents the memory usage from increasing by a factor of m . The encoding process in the Btfy algorithm involves only a single recursive call, which means that multiplication is performed only once. At each layer, the values multiplied with the polynomial are fixed and given by $v_{m/2-\ell_m+1}, \dots, v_{m/2}$. Furthermore, the input and output lie in $\mathbb{F}_2^n$ and $\mathbb{F}_{2^m}^{n/m}$, respectively, both having the same bit length of n . Therefore, the encoding process can be represented as a matrix-vector multiplication, which avoids an increase in memory usage.

Algorithm 4 Polynomial multiplication using FAFFT

Require: $A(x) = a_0 + a_1x + \dots + a_{n-1}x^{n-1} \in \mathbb{F}_{2^m}[x]/\mathcal{P}_l(x)$,

$B(x) = b_0 + b_1x + \dots + b_{n-1}x^{n-1} \in \mathbb{F}_{2^m}[x]/\mathcal{P}_l(x)$.

Ensure: $C(x) = c_0 + c_1x + \dots + c_{2n-2}x^{2n-2} \in \mathbb{F}_{2^m}[x]/\mathcal{P}_l(x)$.

- 1: $A(X) \leftarrow \text{BASISCVT}(A(x))$.
 - 2: $f(X) \leftarrow \text{ENCODE}(A(X))$.
 - 3: $(\hat{a}_0, \dots, \hat{a}_{2^l-1}) \leftarrow \text{BTFY}(f(X), v_{k+m/2})$.
 - 4: $B(X) \leftarrow \text{BASISCVT}(B(x))$.
 - 5: $g(X) \leftarrow \text{ENCODE}(B(X))$.
 - 6: $(\hat{b}_0, \dots, \hat{b}_{2^k-1}) \leftarrow \text{BTFY}(g(X), v_{k+m/2})$.
 - 7: $\hat{C} \leftarrow (\hat{c}_0 = \hat{a}_0 \cdot \hat{b}_0, \dots, \hat{c}_{2^k-1} = \hat{a}_{2^k-1} \cdot \hat{b}_{2^k-1})$.
 - 8: $h(X) \leftarrow \text{INVBTFY}(\hat{C}, v_{k+m/2})$.
 - 9: $C(X) \leftarrow \text{DECODE}(h(X))$.
 - 10: $C(x) \leftarrow \text{INVBASISCVT}(C(X))$.
 - 11: **return** $\text{BTFY}(g(X), 0)$
-

3 Proposed method for optimizing FAFFT for HQC

In this section, we propose the hybrid FAFFT-Karatsuba method that applies FAFFT more efficiently for the HQC polynomial degrees. FAFFT can be applied only to polynomials of degree $2^l - 1$. To ensure that reduction does not affect the result, one chooses the smallest l such that $2n - 2 < 2^l$. For the degree of polynomials on HQC $n = 2^d + r$, this condition leads to $l = d + 2$. Consistent with this rule, the state-of-the-art implementation [LJKH25] selects $2^l = 65536, 131072$, and 131072 for HQC-1, -3, and -5, respectively, as shown in Table 1.

Table 1: FAFFT parameters for HQC used in [LJKH25].

Parameter set	n	2^l	m	k	Σ
HQC-1	17669	65536	32	11	$v_{27} + W_{11}$
HQC-3	35851	131072	32	12	$v_{28} + W_{12}$
HQC-5	57637	131072	32	12	$v_{28} + W_{12}$

Consequently, FAFFT is performed on polynomials whose degree is at least twice and less than four times the original degree. In particular, HQC-1, -3 have $n = 2^{14} + 1285$ and $n = 2^{15} + 3083$, which are slightly larger than powers of two. This mismatch introduces substantial overhead, and as a result HQC-3 uses the same 2^l as HQC-5. The observation

in [LJKH25] that the polynomial-multiplication performance of HQC-3 and HQC-5 is very similar supports this explanation. To address this, we utilize the Hybrid FAFFT–CRT method and the Hybrid FAFFT–Karatsuba method. These methods align large subproducts with a power of two for FAFFT evaluation, while handling the small residual part separately. This allows FAFFT to be applied more efficiently to the polynomial degrees used in HQC. Lastly, to further optimize these FAFFT-based implementations, we improve the butterfly operation in FAFFT by shortening the XOR sequences in fixed-coefficient multiplications and carefully scheduling operations to fully saturate the 14 general-purpose registers available on the ARM Cortex-M4.

3.1 The hybrid FAFFT–CRT method

In the context of HQC, polynomial multiplication requires computing $C(x) = A(x) \cdot B(x)$ in $\mathbb{F}_2[x]$. As discussed in Section 2.2.1, for the polynomial multiplication of degree $n = 2^d + r$ with $r > 0$, the previous approach using FAFFT operates over the ring $\mathbb{F}_2[x]/\mathcal{P}_{d+2}(x)$. For HQC-1 ($n = 17,669$), the previous approach typically employs $\mathcal{P}_{16}(x)$ of degree $2^{16} = 65,536$. This results in significant padding overhead since the required product length is only about 35,338.

To address this inefficiency, we apply the Hybrid FAFFT–CRT method that applies the Chinese Remainder Theorem (CRT) to compute the product over a modulus $\mathcal{P}_{d+1}(x) \cdot x^{r_c}$. Here, $\mathcal{P}_{d+1}(x)$ corresponds to a FAFFT instance, and x^{r_c} covers the residual part. Since $\mathcal{P}_{d+1}(x)$ is not divisible by x , the moduli $\mathcal{P}_{d+1}(x)$ and x^{r_c} are coprime, allowing unique reconstruction.

For HQC-1 such that $d = 14$, instead of using $\mathcal{P}_{16}(x)$, we select the smaller modulus $\mathcal{P}_{15}(x)$ of degree $2^{15} = 32,768$. To ensure the total ring size covers the linear product without reduction, we choose $r_c = 2,592$ as a multiple of 32 for alignment. The condition for correctness is satisfied as:

$$2n - 2 < \deg(\mathcal{P}_{15}(x)) + r_c \implies 35,336 < 32,768 + 2,592 = 35,360.$$

The computation is split into two parts:

$$C_1(x) = A(x)B(x) \pmod{\mathcal{P}_{15}(x)}, \quad C_2(x) = A(x)B(x) \pmod{x^{r_c}}.$$

$C_1(x)$ is computed efficiently using FAFFT of length 2^{15} , and $C_2(x)$ is computed using radix-16 polynomial multiplication. The final result is reconstructed using the explicit CRT formula. First, we compute the correction term $t(x)$ modulo x^{r_c} :

$$t(x) = (C_1(x) \pmod{x^{r_c}} + C_2(x)) \cdot \mathcal{P}_{15}^{-1}(x) \pmod{x^{r_c}}.$$

Here, $\mathcal{P}_{15}^{-1}(x)$ is the modular inverse of $\mathcal{P}_{15}(x)$ modulo x^{r_c} . Finally, the product $C(x)$ is recovered as:

$$C(x) = C_1(x) + t(x) \cdot \mathcal{P}_{15}(x).$$

The final multiplication $t(x) \cdot \mathcal{P}_{15}(x)$ is performed efficiently by taking advantage of the sparsity of $\mathcal{P}_{15}(x)$. Although $\mathcal{P}_{15}(x)$ has a degree of 32,768, it contains only 501 non-zero terms in our parameter set, making it significantly sparse compared to a random dense polynomial. In standard HQC implementations, optimized sparse-dense multiplication is generally avoided because the sparse polynomial usually represents a secret key or error vector. Thus, the operation requires constant-time execution to prevent timing attacks. However, $\mathcal{P}_{15}(x)$ is a fixed and public polynomial determined by the FAFFT parameters. Therefore, we can employ a fast sparse-dense multiplication algorithm without the risk of leaking sensitive information.

For HQC-5 ($n = 57,637$), the required CRT residual degree must satisfy $r_c \geq 49,736$. Consequently, applying the hybrid FAFFT–CRT method to HQC-5 necessitates at least

a FAFFT of size 65,536 and a polynomial multiplication of degree r_c . To verify the feasibility of this approach, we estimated a conservative lower bound on the required clock cycles by summing the cycle counts of the FAFFT of size 65,536 and a polynomial multiplication of degree 35,851 (taken from our HQC-3 benchmark). Since the degree 35,851 is strictly smaller than the required r_c and we explicitly exclude additional overheads, such as $t(x) \cdot \mathcal{P}_{16}(x)$, this estimate represents a strictly lower bound. Even this lower bound amounts to 7,448,930 clock cycles, which already exceeds the 6,290,039 cycles achieved by the state-of-the-art implementation [LJKH25]. Therefore, the hybrid FAFFT-CRT method offers no performance benefit for HQC-5.

Table 2: Parameters for Hybrid FAFFT–CRT method in HQC-1, -3. $\omega(\mathcal{P}_i)$ denotes the Hamming weight of the $\mathcal{P}_i(x)$.

Parameter set	n	FAFFT for C_1				CRT	
		$2^{l'}$	m	k	Σ	r_c	$\omega(\mathcal{P}_i)$
HQC-1	17669	32768	32	10	$v_{26} + W_{10}$	2592	501
HQC-3	35851	65536	32	11	$v_{27} + W_{11}$	6176	473

3.1.1 Efficient sparse-dense polynomial multiplication

The final step of the CRT reconstruction requires computing the product $t(x) \cdot \mathcal{P}_{d+1}(x)$. Since $\mathcal{P}_{d+1}(x)$ is a sparse polynomial and $t(x)$ is a dense polynomial, we employ an optimized shift-and-XOR strategy tailored to the 32-bit word architecture of the ARM Cortex-M4. As previously noted, since $\mathcal{P}_{d+1}(x)$ is a fixed and public polynomial, utilizing its sparsity does not incur side-channel leakage. As detailed in Algorithm 5, we optimize this sparse-dense polynomial multiplication by introducing two key strategies: index grouping and batch processing.

Index grouping A naive implementation would iterate through every non-zero term x^ι of $\mathcal{P}_{d+1}(x)$ and accumulate $t(x) \cdot x^\iota$. This approach requires shifting the dense polynomial $t(x)$ by an arbitrary bit index ι . On the ARM Cortex-M4, performing a bitwise shift on the polynomial is not a single instruction. It requires a sequence of a bitwise shift and a logical OR-shift (utilizing the barrel shifter) for every word of the dense polynomial. To reduce these costs, we minimize bitwise shift operations by grouping the indices of non-zero terms based on their offsets within a 32-bit word. Let $I_{\mathcal{P}_{d+1}}$ denote the set of indices corresponding to the exponents of the non-zero terms in $\mathcal{P}_{d+1}(x)$. We decompose each index $\iota \in I_{\mathcal{P}_{d+1}}$ into a quotient q_ι and a bitwise shift amount s such that $\iota = 32 \cdot q_\iota + s$, where $0 \leq s < 32$. Based on this decomposition, the multiplication can be reformulated as:

$$t(x) \cdot \mathcal{P}_{d+1}(x) = \sum_{\iota \in I_{\mathcal{P}_k}} t(x) \cdot x^\iota = \sum_{s=0}^{31} \left((t(x) \cdot x^s) \cdot \sum_{\substack{\iota \in I_{\mathcal{P}_k} \\ \iota \equiv s \pmod{32}}} (x^{32})^{q_\iota} \right).$$

In our implementation, we group the indices of $\mathcal{P}_{d+1}(x)$ by their bitwise shift amount s . The process is as follows:

1. We compute the bit-shifted dense polynomial $t_s(x) = t(x) \cdot x^s$ within registers, reusing it for all terms with the same shift amount s .
2. For every term in the group with shift amount s , we accumulate $t_s(x)$ shifted by q_ι words. Since word-level shifts correspond to simple memory pointer adjustments, they incur negligible computational cost.

This optimization reduces the number of bitwise shift operations required, significantly accelerating the sparse-dense multiplication.

Batch processing To further optimize performance, we process consecutive words of the dense polynomial in batches per iteration, rather than iterating word-by-word. This strategy provides two advantages:

- **Reduction of load and shift overhead:** Computing the bit-shifted result for a batch of α target words requires loading (Line 8) and shifting (Line 13) $\alpha + 1$ words: the α target words $(v_1, \dots, v_\alpha)$ and the preceding word (v_0) which supplies the carry bits. Consequently, the number of load and shift operations required per generated output word is proportional to $(\alpha + 1)/\alpha$, which decreases as the batch size α increases.
- **Utilization of LDM and STM:** Handling consecutive words enables the use of LDM (Load Multiple) and STM (Store Multiple) instructions. These instructions minimize the instruction fetch overhead and maximize memory bandwidth utilization compared to executing a sequence of single LDR/STR instructions.

In our specific implementation on the ARM Cortex-M4, we adopted a 5-word batch configuration ($\alpha = 5$). The Cortex-M4 provides 14 usable GP registers (R0-R12 and LR). Our innermost loop (lines 15-18) requires four registers to store the result array base address $\&R[i]$, the pointer ptr , the offset off , and the loop counter j . To maximize throughput of this loop using LDM/STM, we must allocate 5 registers to hold the pre-loaded dense polynomial batch $(v_1, \dots, v_5)$ and 5 additional registers as a result polynomial batch $R[i + off, \dots, i + off + 4]$ for the accumulation (load, XOR, store) process. This requires a total of 14 registers, which efficiently uses all of the available GP registers and prevents unnecessary memory accesses. Note that the carry word v_0 is preserved on the stack as it is not required for the XOR accumulation step.

Algorithm 5 outlines the detailed procedure of the proposed sparse-dense polynomial multiplication. To efficiently manage the indices of the sparse polynomial $\mathcal{P}_{d+1}(x)$, the algorithm uses two pre-computed data arrays. The sorted index array (Idx) stores the word offsets (q_ℓ) of the non-zero terms, rearranged such that all terms having the same bit offset $0 \leq s < 32$ are grouped continuously. Complementing this, the count array (Cnt) records the number of non-zero terms associated with each bit offset s . Specifically, for a given bit offset s , the elements of Idx in the range

$$Idx \left[\sum_{k=0}^{s-1} Cnt[k], \sum_{k=0}^s Cnt[k] - 1 \right]$$

correspond exactly to the set of word offsets $\{q_\ell \mid 32 \cdot q_\ell + s \in I_{\mathcal{P}_{d+1}}\}$.

The algorithm takes the dense polynomial array t and these pre-sorted structures as inputs. It iterates through the dense polynomial in 5-word batches. For each batch, the inner loop iterates through shift amounts s from 0 to 31, handling all sparse terms that require a bitwise shift of exactly s . Within this loop, the working registers $\{v_1, \dots, v_5\}$ are updated via a 1-bit left shift (linked with v_0) to align with the current bit index. Subsequently, utilizing $Cnt[s]$ and the sequential segment of Idx , the bit-shifted values are XORed into the corresponding positions of the result array R , minimizing redundant operations.

Algorithm 5 Optimized Sparse-Dense Polynomial Multiplication

Require: R (result array), t (dense polynomial array), Idx (sorted word offsets), Cnt (term counts per shift)

Ensure: $R \leftarrow R + t(x) \cdot \mathcal{P}_{d+1}(x)$

```

1: for  $i = 0$  to  $\text{len}(t) - 5$  step 5 do
2:   /* Load 5 data words ( $v_1 \dots v_5$ ) and 1 carry word ( $v_0$ ) */
3:   if  $i = 0$  then
4:      $v_0 \leftarrow 0$ 
5:   else
6:      $v_0 \leftarrow t[i - 1]$ 
7:   end if
8:    $(v_5, v_4, v_3, v_2, v_1) \leftarrow (t[i + 4], t[i + 3], t[i + 2], t[i + 1], t[i])$ 
9:    $ptr \leftarrow Idx$ 
  /* Unified Shift and XOR Processing */
10:  for  $s = 0$  to 31 do
11:    if  $s > 0$  then
12:      /* Shift chain including carry  $v_0$  */
13:       $(v_5, v_4, v_3, v_2, v_1, v_0) \leftarrow (v_5, v_4, v_3, v_2, v_1, v_0) \ll 1$ 
14:    end if
15:    for  $j = 0$  to  $Cnt[s] - 1$  do
16:       $off \leftarrow *ptr++$ 
17:       $R[i + off, \dots, i + off + 4] \oplus = (v_1, v_2, v_3, v_4, v_5)$ 
18:    end for
19:  end for
20: end for

```

Note: Remaining words processed sequentially if $\text{len}(t) \neq 0 \pmod{5}$.

3.2 The hybrid FAFFT–Karatsuba method

For HQC-1 and HQC-3, we write the degree parameter as $n = 2^d + r$ with $0 < r < 2^d$ and split the inputs at 2^d :

$$A(x) = A_0(x) + x^{2^d} A_1(x), \quad B(x) = B_0(x) + x^{2^d} B_1(x),$$

where $\deg A_0, \deg B_0 < 2^d$ and $\deg A_1, \deg B_1 < r$. Then we compute

$$C_0 = A_0 B_0, \quad C_1 = (A_0 + A_1)(B_0 + B_1), \quad C_2 = A_1 B_1.$$

Finally, the result of multiplication is

$$C(x) = A(x)B(x) = C_0 + x^{2^d} (C_0 + C_1 + C_2) + x^{2 \cdot 2^d} C_2.$$

We compute C_0 and C_1 using FAFFT with $2^{l'} = 2^{d+1}$, and C_2 using radix-16 multiplication.

Algorithm 6 Hybrid FAFFT–Karatsuba method

Require: $A(x) = A_1 X + A_0$, $B(X) = B_1 X + B_0$ for $X = x^{2^{l_0}}$

Ensure: $C(x) = A(x) \times B(x)$

- 1: $C_0 \leftarrow A_0 \times B_0$ using FAFFT algorithm.
- 2: $C_2 \leftarrow A_1 \times B_1$ using radix-16 algorithm.
- 3: $C_1 \leftarrow (A_0 + A_1) \times (B_0 + B_1)$ using FAFFT algorithm.
- 4: $C(x) \leftarrow C_2 X^2 + (C_0 + C_1 + C_2) X + C_0$.
- 5: **return** $C(x)$

In our approach, we choose $X = x^{2^d}$ as the split point. While one could split the polynomials at $X = x^r$, which would only require modifying lines 1 and 2 of Algorithm 6, splitting at $X = x^{2^d}$ is more efficient for the following reason.

Let $X = x^{2^d}$. Given the degrees $\deg A_0 = \deg B_0 = 2^d - 1$ and $\deg A_1 = \deg B_1 = r - 1$, the Karatsuba components have degrees $\deg C_0 = \deg C_1 = 2 \cdot 2^d - 2$ and $\deg C_2 = 2r - 2$. With the split point $X = x^{2^d}$, the degrees of the individual terms in the final result $C(x) = C_0 + (C_0 + C_1 + C_2)X + C_2X^2$ relate to the total degree $2n - 2$ as follows:

$$\deg C_0 = 2 \cdot 2^d - 2 < 2n - 2, \quad \deg(C_2X^2) = 2r + 2 \cdot 2^d - 2 = 2n - 2.$$

Since the total degree is $\deg C(x) = 2n - 2$, the degree of the middle term must satisfy:

$$\deg((C_0 + C_1 + C_2)X) \leq 2n - 2$$

$$\deg(C_0 + C_1 + C_2) \leq 2n - 2 - 2^d = 2^d + 2r - 2.$$

If the condition $2r < 2^d$ is satisfied, which holds for both HQC-1 and HQC-3, we obtain the following inequality:

$$\deg(C_0 + C_1 + C_2) \leq 2^d + 2r - 2 < 2 \cdot 2^d - 2 = \max\{\deg C_0, \deg C_1, \deg C_2\}. \quad (1)$$

Note that $2 \cdot 2^d - 2$ is the maximum degree supported by the FAFFT of length 2^{d+1} used to compute C_0 and C_1 . Consequently, in line 4 of Algorithm 6, when computing the sum $C_0 + C_1 + C_2$, the upper $2^d - 2r$ bits do not need to be calculated or XORed, as they are guaranteed to be zero.

In contrast, if the polynomials are split at $X = x^r$, the degree of the middle term is $\deg(C_0 + C_1 + C_2) = 2 \cdot 2^d + r - 2$. Since this exceeds $2 \cdot 2^d - 2$, the inequality in (1) does not hold. Thus, splitting at $X = x^{2^d}$ reduces the XOR operations required for reconstruction.

Table 3: parameters for FAFFT with Karatsuba in HQC-1, -3.

Parameter set	n	FAFFT for C_0, C_1				radix-16 for C_2
		$2^{l'}$	m	k	Σ	r
HQC-1	17669	32768	32	10	$v_{26} + W_{10}$	1285
HQC-3	35851	65536	32	11	$v_{27} + W_{11}$	3083

3.3 Optimizing butterfly algorithm of FAFFT

During the butterfly stage, the term $s_i(\alpha) \cdot p_1(X)$ is evaluated as follows [LJKH25]. For $m = 32$, write $s_i(\alpha) = w + v$ with $w \in W_k$ and $v \in \{v_{17}, \dots, v_{k+16}\}$. Then $s_i(\alpha) \cdot p_1(X) = w \cdot p_1(X) + v \cdot p_2(X)$. Since $k < 16$, w lies in a proper subspace $\mathbb{F}_2^{m'} \subsetneq \mathbb{F}_2^{32}$ with $m' \leq 16$. Consequently, the product $w \cdot p_1(X)$ over $\mathbb{F}_2^{32}$ can be computed as $(32/m')$ products over $\mathbb{F}_2^{m'}$. For the term $v \cdot p_2(X)$, [LJKH25] exploits that $v \in \{v_{17}, \dots, v_{k+16}\}$ is fixed, converts the fixed part into a matrix-vector multiplication, and then searches XOR sequences for this operation using the method of [XZL⁺20]. This yields a 23-44% reduction in XORs compared with the previous implementation [CCK21].

We searched XOR sequences that use fewer XORs than [LJKH25]. Moreover, by spilling the input and output registers to the stack and scheduling with all 14 GP registers and 32 FP registers of Cortex-M4, we achieved a more efficient register allocation, which reduces `vmov` instructions. Table 4 presents a comparison of XOR and instruction counts against [LJKH25]. For v_{24} , our sequence uses more XORs, but it is more suitable for register scheduling, which reduces the overall instruction count. Since performance depends on the instruction count rather than the XOR count, we select the implementation with fewer instructions.

Table 4: XOR and instruction counts for multiplication by v_i .

Multiplier	[LJKH25]		Ours	
	XOR	clock cycle	XOR	clock cycle
v_{17}	161	485	159	462
v_{18}	153	465	153	440
v_{19}	171	515	166	479
v_{20}	173	511	172	485
v_{21}	172	514	166	483
v_{22}	165	501	160	469
v_{23}	163	497	163	476
v_{24}	178	548	180	503
v_{25}	189	555	180	501
v_{26}	190	562	179	516
v_{27}	179	539	173	538
v_{28}	187	559	183	518

In addition, we optimize the swap operations that occur during the butterfly using the **SWAPMOVE** subroutine proposed in [MPC00]. The **SWAPMOVE** operation applies a mask to select specific bits from register R_2 and swaps them with the bits of register R_1 shifted by n . We denote it as **SWAPMOVE**(R_1, R_2, mask, n). On the Cortex-M4, this operation can be implemented using four instructions. Algorithm 7 represents the **SWAPMOVE** subroutine.

Algorithm 7 SWAPMOVE

Require: R_1, R_2, mask, n

Ensure: bit swap between the R_1, R_2

- 1: EOR tmp, $R_2, R_1, \text{LSR } n$
 - 2: AND tmp, tmp, mask
 - 3: EOR R_2, R_2, tmp
 - 4: EOR $R_1, R_1, \text{tmp, LSL } n$
-

Our butterfly is implemented in a bitsliced manner on a Cortex-M4 with 32-bit registers. In the final five layers of Algorithm 2, the offset n between the data corresponding to $p_0(X)$ and $p_1(X)$ is at most 16. In the bitsliced representation, the two slices reside within the same 32-bit word. For this reason, bit swaps in registers are required. There are $2^l/32$ such swaps for each layer, so a single FAFFT multiplication performs $2^l/32 \cdot 5 \cdot 2$ swaps in total. [LJKH25] implements this swap as Algorithm 8.

Algorithm 8 Bit swap in the butterfly stage [LJKH25]

Require: $\text{data}[i+j], \text{data}[i+32+j]$

Ensure: $\text{data}[i+j], \text{data}[i+32+j]$ s.t. the bits selected by MASK are exchanged between the two words with shift offset n .

- 1: uint32_t data_l = ($\text{data}[i+j] \& \sim \text{MASK}$) | (($\text{data}[i+32+j] \& \sim \text{MASK}$) $\ll n$);
 - 2: uint32_t data_h = (($\text{data}[i+j] \& \text{MASK}$) $\gg n$) | ($\text{data}[i+32+j] \& \text{MASK}$);
 - 3: $\text{data}[i+j] = \text{data}_l$;
 - 4: $\text{data}[i+32+j] = \text{data}_h$;
-

The implementation in Algorithm 8 requires eight instructions. Using **SWAPMOVE**, the same operation can be realized with four instructions as Algorithm 9, reducing four instructions for each swap.

```

1: uint32_t tmp = data[i+32+j] ^ (data[i+j] >> n);
2: tmp = tmp & MASK;
3: data[i+32+j] = data[i+32+j] ^ tmp;
4: data[i+j] = data[i+j] ^ (tmp << n);

```

4 Radix-16 multiplicaton

In [CC21], a radix-16 representation for degree-7 polynomials over $\mathbb{F}_2[x]$ was introduced. Given

$$A_0(x) = a_0 + a_1x + \cdots + a_7x^7.$$

and define its radix-16 representation as the 32-bit integer

$$A'_0 = a_0 + a_1 2^4 + a_2 2^8 + \cdots + a_7 2^{28}.$$

Under this encoding, the product of two polynomials $A(x)$ and $B(x)$ can be computed by a single integer multiplication:

$$P_{0,0} = A'_0 \times B'_0 = a_0 b_0 + (a_0 b_1 + a_1 b_0) 2^4 + \cdots + (a_6 b_7 + a_7 b_6) 2^{52} + a_7 b_7 2^{56}$$

Since for each coefficient index k the sum $\sum_{i+j=k} a_i b_j$ contains at most seven terms (each equal to 0 or 1), no partial sum exceeds $8 < 16$, so no carries occur across 4-bit boundaries. Therefore, extracting the bits at positions 0, 4, 8, $\dots$, 56 from the 64-bit product recovers exactly the 15 coefficients of $A_0(x) \times B_0(x)$ in $\mathbb{F}_2[x]$. Figure 1 represents the 32-bit integer multiplication to compute 8-bit $\mathbb{F}_2[x]$ polynomial multiplication under the radix-16 representation.

[illegible]

Figure 1: 8-bit $\mathbb{F}_2[x]$ polynomial multiplication with radix-16 representation.

Using this method, polynomial multiplication can be reduced to integer multiplication. Unlike high-performance architectures such as AVX2 and NEON, embedded systems like Cortex-M4 do not support carry-less multiplication instructions. Therefore, for Cortex-M4, implementing $\mathbb{F}_2[x]$ multiplication with carry-less multiplication via radix-16 multiplication is efficient.

4.1 32-bit base multiplication in radix-16 representation

Using the method introduced in the previous section, 8-bit multiplications over $\mathbb{F}_2[x]$ can be performed. In this section, we describe the technique for 32-bit $\mathbb{F}_2[x]$ multiplication as proposed in [CGKT22].

For given 32-bit $\mathbb{F}_2[x]$ polynomial $A(x) = a_0 + a_1x + \dots + a_{31}x^{31}$ and $B(x) = b_0 + b_1x + \dots + b_{31}x^{31}$, we split each 32-bit polynomial into four 8-bit polynomials:

$$A_i(x) = \sum_{j=0}^7 a_{8i+j} x^j, \quad B_i(x) = \sum_{j=0}^7 b_{8i+j} x^j.$$

Finally, $P_{i,j} = A_i(x) \times B_j(x)$ are summed at their appropriate offsets to compute $C'(x)$, the radix-16 representation of $C(x) = A(x) \times B(x)$. Each multiplication $A_i(x) \times B_j(x)$ is computed via integer multiplication on its radix-16 representation, as shown in Figure 1. We now introduce efficient methods for the radix-16 conversion and the summation of $P_{i,j}$.

4.1.1 Data conversion for the radix-16 representation

Storing each 8-bit polynomial $A_i(x)$ as a 32-bit integer A'_i in a 32-bit register incurs increases register usage by a factor of four. To eliminate this overhead, [CGKT22] proposes an efficient method that converts four 8-bit polynomials into their radix-16 representations within a single 32-bit register. We call it *packed radix-16 representation*.

The conversion method is based on the SWAPMOVE operation. Let each 8-bit polynomial $A_i(x) = \sum_{j=0}^7 a_{i,j}x^j$ be stored in a 32-bit register as

$$[a_{0,0}, \dots, a_{0,7}, \quad a_{1,0}, \dots, a_{1,7}, \quad a_{2,0}, \dots, a_{2,7}, \quad a_{3,0}, \dots, a_{3,7}].$$

Then three SWAPMOVE operations with shifts of 12, 6, and 3 bits reorder these bits in place to

$$[a_{0,0}, a_{2,0}, a_{1,0}, a_{3,0}, \quad a_{0,1}, a_{2,1}, a_{1,1}, a_{3,1}, \quad \dots, \quad a_{0,7}, a_{2,7}, a_{1,7}, a_{3,7}].$$

In total, only 12 instructions are required to construct A'_{pack} , the packed radix-16 representation of A . The full instruction sequence and its use for data conversion are given in Algorithm 10.

Algorithm 10 Data conversion for the radix-16 representation

Require: 32-bit polynomial A

Ensure: A'_{pack} the radix-16 representation of A

- 1: $A'_{pack} \leftarrow A$
 - 2: SWAPMOVE(A'_{pack} , A'_{pack} , 0x0000 f0f0, 12)
 - 3: SWAPMOVE(A'_{pack} , A'_{pack} , 0x00cc 00cc, 6)
 - 4: SWAPMOVE(A'_{pack} , A'_{pack} , 0x0a0a 0a0a, 3)
 - 5: **return** A'_{pack}
-

After this conversion, the 32-bit radix-16 representations for each 8-bit polynomial can be extracted by simple mask-and-shift operations. Let $\text{mask} = 0x1111\ 1111$. Then each of the four radix-16 representations, A'_0 , A'_1 , A'_2 , and A'_3 , can be computed with a single instruction as follows:

$$\begin{aligned} A'_0 &= A'_{pack} \& \text{mask}, \\ A'_1 &= (A'_{pack} \gg 2) \& \text{mask}, \\ A'_2 &= (A'_{pack} \gg 1) \& \text{mask}, \\ A'_3 &= (A'_{pack} \gg 3) \& \text{mask}. \end{aligned}$$

After completing all multiplications in the radix-16 representation, the inverse conversion should be applied to convert the data back to its original format. The inverse conversion can be performed by executing the **SWAPMOVE** sequence in reverse order with shifts of 3, 6, and 12 bits using the same masks. This holds because **SWAPMOVE** is self-inverse, i.e., applying it twice serves as the identity.

4.1.2 Construction of the 32-bit polynomial multiplication

$C'(x)$ which is the radix-16 representation of $C(x)$ can be obtained by summing the sixteen 64-bit integer products $P_{i,k-i} = A'_i \times B'_j$ as Figure 2. Each $P_{i,j}$ contains other set bits outside the positions 0, 4, 8, ..., 56. To ensure correctness, $P_{i,j}$ would be masked with $mask = 0x1111\ 1111\ 1111\ 1111$ via AND operation before accumulation.

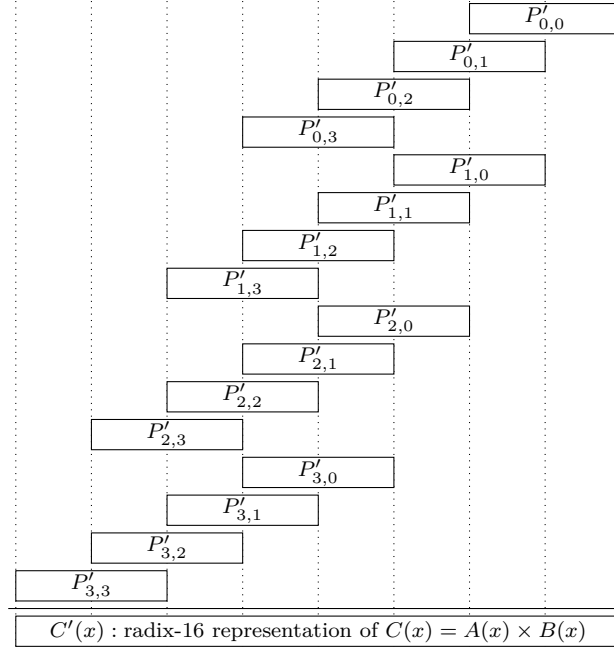


Figure 2: Constructing the 32-bit base multiplication.

However, as shown in figure 1, adding two upper 32-bit halves or upper 32-bit half to lower 32-bit half does not propagate carries beyond 4 bits. Therefore, in these cases, it is not necessary to perform AND with $mask$ before addition, saving one less instruction. In addition, the UMLAL instruction is not applicable to uninitialized registers. If both the upper and lower 32-bit halves to be added are uninitialized, the UMULL instruction must be employed. However, if only one of them is uninitialized, it is necessary to employ the UMLAL instruction after initializing the uninitialized 32-bit half with MOV instruction. The former requires one instruction, while the latter requires two. Consequently, the total number of instructions depends on the order of addition of the $P_{i,j}$ s. Considering all of these and register scheduling, the accumulation operation is performed in the order of Figure 3.

After obtaining $C'(x)$ using the method in Figure 3, it must be converted from the radix-16 representation to the original format. As aforementioned, the conversion from the packed radix-16 representation to the original format is performed by the three **SWAPMOVE**s in Algorithm 10 in reverse order, requiring a total of 12 instructions. To employ this technique, one converts $C'(x)$ to a packed radix-16 representation to get C'_{pack} , then converts to $C(x)$. In summary, 32-bit base multiplication in radix-16 representation is

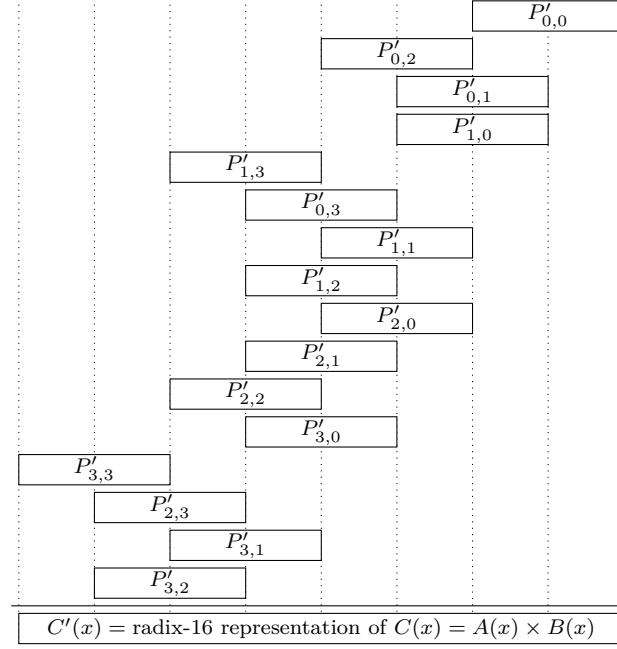


Figure 3: Optimized sequence for constructing the 32-bit base multiplication.

performed as follows:

- **Step 1:** Apply algorithm 10 to $A(x)$ and $B(x)$ to obtain A'_{pack} and B'_{pack} .
- **Step 2:** Extract A'_i and B'_i ($0 \leq i \leq 3$) from A'_{pack} and B'_{pack} .
- **Step 3:** Compute $P_{i,j} = A'_i \times B'_j$ ($0 \leq i, j \leq 3$) by integer multiplication.
- **Step 4:** Accumulate the $P_{i,j}$ in the order of Figure 3 to form $C'(x)$.
- **Step 5:** Construct C'_{pack} by repacking $C'(x)$.
- **Step 6:** Convert C'_{pack} to $C(x)$ by the inverse of Algorithm 10.

4.2 Toom-Cook and Karatsuba algorithms with the radix-16 representation

HQC-1, -3, -5 contain $\mathbb{F}_2[x]$ polynomial multiplications of degree 17668, 35850, and 57636, respectively. The Toom-Cook and Karatsuba algorithms are efficient for polynomial multiplications of these large orders, reducing the complexity of the multiplication. Since both algorithms on $\mathbb{F}_2[x]$ are performed word-wise and all operations are bitwise XOR except for the base multiplication, they can be applied with the radix-16 representation without any modification.

Therefore, before performing the polynomial multiplications, we first convert the polynomial to a radix-16 representation. We then apply the Karatsuba and Toom-Cook algorithms to split the polynomial until it is 32-bit, and perform Steps 2-5 of 4.1 to perform multiplications with inputs and outputs in the radix-16 representation.

5 Proposed method for optimizing the radix-16 multiplication for HQC

In this section, we present a method to optimize radix-16 multiplication by exploring Karatsuba/Toom-Cook combinations. In [CGKT22], the authors employed the `gf2x` library to identify optimal algorithmic combinations. In contrast, our work adopts a theoretical approach based on instruction count, as in [JLK⁺24], to explore optimal combinations. The Cortex-M4 processor lacks features such as out-of-order execution and cache, ensuring that each instruction consistently executes within a fixed number of clock cycles. This makes instruction count a suitable metric for algorithmic optimization. Furthermore, since we optimize under an operation-count cost model rather than hardware-specific autotuning, the resulting optimal combination set is consistent across platforms. Under the assumption that other instructions, such as MOV, LDR, and STR, scale proportionally with the number of XOR and base multiplications, we argue that our approach yields the theoretically most efficient combination. A detailed performance comparison between the `gf2x` library and our optimized combination is provided in Section 5.3. The following describes the search method applied in this paper.

5.1 Karatsuba/Toom-Cook optimization using instruction count

Both the Karatsuba and Toom-Cook algorithms consist of only XOR operations and internal multiplications. The combination of the two algorithms determines the number of XORs and base multiplications. Since the target platform is a Cortex-M4 processor, where operations are performed on 32-bit words, we define an n -word multiplication as a polynomial multiplication between two polynomials represented by n 32-bit words. Based on this definition, [JLK⁺24] finds the optimal combination that minimizes the following cost function:

$$\text{cost}(n) = \text{cnt}_{\text{XOR}}(n) + w_{\text{mul}} \cdot \text{cnt}_{\text{mul}}(n),$$

where $\text{cnt}_{\text{XOR}}(n)$ and $\text{cnt}_{\text{mul}}(n)$ denote the numbers of 32-bit XOR operations and base multiplications, respectively, and w_{mul} is the weight assigned to base multiplications. By definition, for $n = 1$, the values are:

$$\text{cnt}_{\text{XOR}}(1) = 0, \quad \text{cnt}_{\text{mul}}(1) = 1, \quad \text{cost}(1) = w_{\text{mul}}.$$

Let L denote the set of all candidate algorithms under consideration. When computing an n -word multiplication using some $\mathcal{A} \in L$, we denote by α the number of XOR operations performed at the top level, by $n_1, \dots, n_r$ the word-lengths of the internal multiplications, and by $\beta_1, \dots, \beta_r$ the counts of internal multiplications on $n_1, \dots, n_r$ -word polynomials, respectively. Then, the total number of 32-bit XOR operations and base multiplications at the top level for algorithm $\mathcal{A}$, denoted by $\text{cnt}_{\text{XOR}}^{\mathcal{A}}(n)$ and $\text{cnt}_{\text{mul}}^{\mathcal{A}}(n)$, respectively, and the corresponding cost function $\text{cost}^{\mathcal{A}}(n)$ are as follows:

$$\begin{aligned} \text{cnt}_{\text{XOR}}^{\mathcal{A}}(n) &= \alpha + \sum_{i=1}^r \beta_i \text{cnt}_{\text{XOR}}(n_i) \\ \text{cnt}_{\text{mul}}^{\mathcal{A}}(n) &= \sum_{i=1}^r \beta_i \text{cnt}_{\text{mul}}(n_i) \\ \text{cost}^{\mathcal{A}}(n) &= \text{cnt}_{\text{XOR}}^{\mathcal{A}}(n) + w_{\text{mul}} \text{cnt}_{\text{mul}}^{\mathcal{A}}(n) \end{aligned}$$

Let $\mathcal{A}^* = \arg \min_{\mathcal{A} \in L} \text{cost}^{\mathcal{A}}(n)$. Then

$$\text{cnt}_{\text{XOR}}(n) = \text{cnt}_{\text{XOR}}^{\mathcal{A}^*}(n), \quad \text{cnt}_{\text{mul}}(n) = \text{cnt}_{\text{mul}}^{\mathcal{A}^*}(n), \quad \text{cost}(n) = \text{cost}^{\mathcal{A}^*}(n).$$

Table 5: The number of internal multiplications and XORs in the Karatsuba and Toom-Cook algorithms.

Algorithm	Input (words)	# of internal multiplications				XOR
		$(k-1)$ -word	k -word	$(k+1)$ -word	$(k+2)$ -word	
2-Karat	$2k$	0	3	0	0	$7k$
	$2k+1$	0	1	2	0	$7k+3$
3-Karat	$3k$	0	6	0	0	$20k$
	$3k+1$	0	3	3	0	$20k+8$
	$3k+2$	0	1	5	0	$20k+14$
5-Karat	$5k$	0	15	0	0	$80k$
TC3	$3k$	0	3	0	2	$38k+18$
	$3k+1$	1	0	2	2	$38k+22$
	$3k+2$	0	1	2	2	$38k+32$

In this way, the most efficient algorithm in each length can be determined, and the optimal combination of algorithms for multiplication in the desired lengths can be found. In an ideal case where only XORs and base multiplications occur, w_{mul} can be set as the ratio of clock cycles required for the two operations. However, in a real implementation additional, MOV, LDR and STR instructions are required to load operands and store results before and after each operation. Since the number of these operations depends on the number of XORs and base multiplications, w_{mul} must be set to account for this. Consequently, we enumerate all optimal algorithmic combinations for each of w_{mul} and experimentally measure their performance to determine the optimal combinations.

Table 5 lists the number of XOR instructions and internal multiplications performed for each Karatsuba and Toom-Cook algorithm. From this table, $\text{cnt}_{\text{XOR}}(n)$ and $\text{cnt}_{mul}(n)$ can be derived.

5.2 The best combinations for HQC

This section describes the optimal combination of algorithms for polynomial multiplication of HQC-1, -3, and -5 obtained through the methods described in 5.1. For $1 \leq w_{mul} \leq 500$, we explored the optimal combinations for 50,000 cases by increasing w_{mul} in 0.01 increments. This resulted in 4, 6, and 5 combinations for HQC-1, -3, and -5, respectively, depending on the range of w_{mul} .

Table 6 shows the performance on Cortex-M4 for each optimal combination selected by the cost model over ranges of w_{mul} . Minimum clock cycles occurred at $1 \leq w_{mul} \leq 29.00$ (HQC-1), $2.49 \leq w_{mul} \leq 500$ (HQC-3), and $1.60 \leq w_{mul} \leq 5.06$ (HQC-5). The environment of benchmarking is described in 6.

5.3 Comparison with gf2x library

In this section, we compare our method with the **gf2x** library. The **gf2x** library is the state-of-the-art approach for selecting optimal $\mathbb{F}_2[x]$ multiplication methods at arbitrary polynomial degrees. It generates optimized routines for diverse environments and provides a multiplication API. It chooses among candidate algorithms by experimentally measuring and comparing their clock cycles. However, this procedure includes additional operations such as conditional branches and table lookups, which prevent a fair comparison.

For a fair comparison, both approaches must consider the same set of algorithms. In a radix-16 representation, performing shifts within a 32-bit word requires more complex operations [CGKT22]. Among the algorithms considered by **gf2x** library that avoid such shifts, the options are Toom-Cook 3-way (TC3), Toom-Cook 4-way (TC4), and 2-Karatsuba.

Table 6: Performance (clock cycles) of optimal Karatsuba/Toom-Cook combinations across w_{mul} for HQC parameter sets on Cortex-M4.

Parameter set	Case	Weight	Clock cycle
HQC-1	1	$1 \leq w_{\text{mul}} \leq 29.00$	2,145,505
	2	$29.01 \leq w_{\text{mul}} \leq 34.00$	2,178,419
	3	$34.01 \leq w_{\text{mul}} \leq 50.61$	2,201,402
	4	$50.62 \leq w_{\text{mul}} \leq 500$	2,226,850
HQC-3	1	$1 \leq w_{\text{mul}} \leq 1.51$	6,320,177
	2	$1.52 \leq w_{\text{mul}} \leq 1.74$	6,306,667
	3	$1.75 \leq w_{\text{mul}} \leq 1.85$	6,183,733
	4	$1.86 \leq w_{\text{mul}} \leq 2.16$	6,164,280
	5	$2.17 \leq w_{\text{mul}} \leq 2.48$	6,131,652
	6	$2.49 \leq w_{\text{mul}} \leq 500$	6,078,474
HQC-5	1	$1 \leq w_{\text{mul}} \leq 1.59$	12,464,389
	2	$1.60 \leq w_{\text{mul}} \leq 5.06$	12,456,160
	3	$5.07 \leq w_{\text{mul}} \leq 13.66$	12,466,520
	4	$13.67 \leq w_{\text{mul}} \leq 14.00$	12,513,222
	5	$14.01 \leq w_{\text{mul}} \leq 500$	12,460,612

Although TC4 has the best asymptotic complexity, its practical performance is poor, and it does not arise for polynomial degrees up to those of HQC [CGKT22]. Therefore, in both **gf2x** library and our approach, we consider only TC3 and 2-Karatsuba, and we compare clock cycles at the polynomial degrees corresponding to the three HQC parameter sets. Since our objective is to compare the generated algorithmic combinations themselves, we did not rely on the API provided by **gf2x** library. Instead, we implemented the candidate algorithms under identical conditions and compared them directly. As a result, for the polynomial degrees of HQC-1 and HQC-3, both approaches produced the same algorithmic combinations, whereas for HQC-5 they differed. For HQC-5, the combination produced by our method required 21.7% fewer clock cycles than the combination produced by **gf2x** library. Detailed cycle counts are provided in Table 7.

Table 7: Clock cycle comparison on Cortex-M4 for **gf2x** library [BGTZ23] and our method, both restricted to Toom-Cook 3-way and 2-Karatsuba.

Algorithm	gf2x library	Our method
HQC-1	2,636,770	2,636,770
HQC-3	7,576,158	7,576,158
HQC-5	21,985,410	17,204,103

6 Experimental results and comparisons

In this section, we evaluate the efficiency of the optimization methods proposed in Section 3 and Section 5 by implementing and benchmarking them on a NUCLEO-L4R5ZI evaluation board equipped with an STM32L4R5ZI microcontroller. The platform is based on an ARM Cortex-M4 core and features 2 MB of flash memory and 640 KB of SRAM. As specified by the **pqm4** framework [KPR⁺], we set the system clock to 20 MHz to achieve zero wait states, and we compile all code with **arm-none-eabi-gcc** version 11.3.1.

6.1 FAFFT butterfly

Table 8 compares the performance of the optimized FAFFT butterfly described in Section 3.3 against the implementation in [LJKH25]. Our optimized butterfly reduces the cycle count by 3.2% to 3.5% across all lengths used in HQC.

Table 8: Clock cycle comparison of the FAFFT butterfly.

2^l	[LJKH25]	Our code
32768	314,494	303,740 (-3.4%)
65536	709,110	683,954 (-3.5%)
131072	1,589,934	1,539,345 (-3.2%)

6.2 Polynomial multiplication

Table 9 presents the benchmark results for various polynomial multiplication methods. In this comparison, the FAFFT column of our code refers to the implementation from [LJKH25] with our butterfly optimization applied.

For HQC-1 and HQC-3, the Hybrid FAFFT-CRT method achieves the highest efficiency. Specifically, for HQC-1, the FAFFT-CRT method reduces the cycle count by 33.4% compared to the baseline. For HQC-3, it results in a 27.2% reduction. While the Hybrid FAFFT-Karatsuba and Radix-16 methods also outperform the baseline for these parameters, they are less efficient than the FAFFT-CRT approach.

For HQC-5, where hybrid splitting methods are not applied, our FAFFT code with our optimized butterfly algorithm reduces the cycle count by 3.0% compared to the baseline. The Radix-16 method is significantly less efficient for this degree due to the increased complexity of Karatsuba and Toom-Cook combinations required for the larger degree.

Table 9: Clock cycle comparison of HQC polynomial multiplication.

Param.	n	[LJKH25]	Our code			
			FAFFT	FAFFT-CRT	FAFFT-Karat	Radix-16
HQC-1	17,669	2,841,783	2,756,150	1,891,458	2,521,564	2,147,797
HQC-3	35,851	6,279,452	6,092,539	4,572,399	5,697,162	6,083,163
HQC-5	57,637	6,290,041	6,103,141	-	-	12,460,424

6.3 Application to HQC

In this section, we evaluate the performance of the three HQC parameter sets across five distinct implementations: (i) the baseline FAFFT from [LJKH25], (ii) the FAFFT with our optimized butterfly algorithm, (iii) the hybrid FAFFT-CRT method, (iv) the hybrid FAFFT-Karatsuba method, and (v) the radix-16 implementation. Notably, both hybrid FAFFT approaches also integrate the optimized butterfly algorithm. The benchmark results are summarized in Table 10, where **Keygen**, **Encap**, and **Decap** denote key generation, encapsulation, and decapsulation, respectively.

For HQC-1, our FAFFT-CRT method improves the performance of key generation, encapsulation, and decapsulation by 20.5%, 19.9%, and 18.4%, respectively, compared to the baseline. In the case of HQC-3, the same method provides improvements of 16.7%, 16.5%, and 15.8%. For HQC-5, where hybrid methods are not applied, our optimized FAFFT achieves a consistent improvement of approximately 1.3%. Conversely, the Radix-16 method at this security level is slower by 48-51%, which is consistent with the asymptotic advantage of FFT-based multiplication for larger polynomial degrees.

Table 10: Clock cycles for HQC key generation, encapsulation, and decapsulation on Cortex-M4.

Algorithm		[LJKH25]	Our code			
			FAFFT	FAFFT-CRT	FAFFT-Karat	Radix-16
HQC-1	Keygen	4,015,835	3,930,193	3,065,459	3,692,595	3,322,032
	Encap	7,843,285	7,671,995	5,942,521	7,196,807	6,455,688
	Decap	13,026,321	12,769,358	10,175,173	12,056,601	10,944,987
HQC-3	Keygen	9,497,097	9,310,200	7,790,316	8,914,845	9,300,408
	Encap	18,341,826	17,968,035	14,928,198	17,177,322	17,948,454
	Decap	28,863,458	28,302,758	23,743,111	27,116,701	28,273,407
HQC-5	Keygen	12,700,457	12,513,535	–	–	18,874,866
	Encap	24,109,313	23,735,473	–	–	36,458,054
	Decap	38,752,685	38,191,910	–	–	57,275,806

7 Conclusion

In this paper, we present several optimization techniques for HQC polynomial multiplication on the ARM Cortex-M4, including hybrid FAFFT-Karatsuba, hybrid FAFFT-CRT, and radix-16 methods.

The hybrid FAFFT-Karatsuba and FAFFT-CRT methods are particularly efficient for polynomial degrees of the form $n = 2^d + r$ with a small r . These methods enable the multiplication to be performed using an FAFFT of size 2^{d+1} , whereas the previous approach requires a size 2^{d+2} transform. Furthermore, we improved the FAFFT butterfly through efficient register scheduling, shortened XOR sequences for fixed-coefficient multiplications, and the replacement of bit swaps with the `SWAPMOVE` instruction.

Radix-16 multiplication is an efficient technique for environments like the Cortex-M4 that do not support dedicated carry-less multiplication instructions but provide Multiply-Accumulate (MAC) instructions. It accelerates the computation by performing carry-less multiplication through integer multiplication. We apply radix-16 multiplication to HQC for the first time, to the best of our knowledge. We further refined this approach by identifying more efficient Karatsuba and Toom-Cook combinations through an operation-count cost model.

Experimental results on the NUCLEO-L4R5ZI board show that our methods improve the state-of-the-art implementation of HQC for cortex-M4. The hybrid FAFFT-CRT method achieved the highest efficiency for HQC-1 and HQC-3. For HQC-1, it reduces the cycle counts for key generation, encapsulation, decapsulation by 23.7%, 24.2%, 21.9%, respectively, and for HQC-3, it reduces them by 18.0%, 18.6%, and 17.7%, respectively. For HQC-5, where hybrid methods are not suitable for the given degree, our optimized butterfly still provided a performance gain of 1.4 - 1.6%. As HQC is selected for NIST standardization, our optimizations enable its practical deployment on resource-constrained embedded platforms.

References

- [AAB⁺22] Carlos Aguilar-Melchor, Nicolas Aragon, Slim Bettaieb, Loïc Bidoux, Olivier Blazy, Jean-Christophe Deneuville, Philippe Gaborit, Edoardo Persichetti, Gilles Zémor, Jurjen Bos, Arnaud Dion, Jerome Lacan, Jean-Marc Robert, and Pascal Veron. HQC. Technical report, National Institute of Standards and Technology, 2022. available at <https://csrc.nist.gov/Projects/post-quantum-cryptography/round-4-submissions>.

- [ABCG20] Erdem Alkim, Yusuf Alper Bilgin, Murat Cenk, and François Gérard. Cortex-M4 optimizations for $\{R,M\}$ LWE schemes. *IACR TCHES*, 2020(3):336–357, 2020.
- [ACC⁺22] Amin Abdulrahman, Jiun-Peng Chen, Yu-Jia Chen, Vincent Hwang, Matthias J. Kannwischer, and Bo-Yin Yang. Multi-moduli NTTs for Saber on Cortex-M3 and Cortex-M4. *IACR TCHES*, 2022(1):127–151, 2022.
- [AHKS22] Amin Abdulrahman, Vincent Hwang, Matthias J. Kannwischer, and Amber Sprenkels. Faster Kyber and Dilithium on the Cortex-M4. In Giuseppe Ateniese and Daniele Venturi, editors, *ACNS 22International Conference on Applied Cryptography and Network Security*, volume 13269 of *LNCS*, pages 853–871. Springer, Cham, June 2022.
- [BGTZ23] R. P. Brent, P. Gaudry, E. Thomé, and P. Zimmermann. gf2x library. <https://gitlab.inria.fr/gf2x/gf2x>, 2023. Accessed: Feb. 11, 2023. [Online].
- [BKS19] Leon Botros, Matthias J. Kannwischer, and Peter Schwabe. Memory-efficient high-speed implementation of Kyber on Cortex-M4. In Johannes Buchmann, Abderrahmane Nitaj, and Tajje eddine Rachidi, editors, *AFRICACRYPT 19*, volume 11627 of *LNCS*, pages 209–228. Springer, Cham, July 2019.
- [Can89] David G Cantor. On arithmetical algorithms over finite fields. *Journal of Combinatorial Theory, Series A*, 50(2):285–300, 1989.
- [CC21] Ming-Shing Chen and Tung Chou. Classic McEliece on the ARM Cortex-M4. *IACR TCHES*, 2021(3):125–148, 2021.
- [CCK21] Ming-Shing Chen, Tung Chou, and Markus Krausz. Optimizing BIKE for the intel haswell and ARM Cortex-M4. *IACR TCHES*, 2021(3):97–124, 2021.
- [CCPY26] Ming-Shing Chen, Chun-Ming Chiu, Chun-Tao Peng, and Bo-Yin Yang. Accelerating HQC with additive FFT. Cryptology ePrint Archive, Paper 2026/014, 2026.
- [CGKT22] Ming-Shing Chen, Tim Güneysu, Markus Krausz, and Jan Philipp Thoma. Carry-less to BIKE faster. In Giuseppe Ateniese and Daniele Venturi, editors, *ACNS 22International Conference on Applied Cryptography and Network Security*, volume 13269 of *LNCS*, pages 833–852. Springer, Cham, June 2022.
- [CHK⁺21] Chi-Ming Marvin Chung, Vincent Hwang, Matthias J. Kannwischer, Gregor Seiler, Cheng-Jhih Shih, and Bo-Yin Yang. NTT multiplication for NTT-unfriendly rings. *IACR TCHES*, 2021(2):159–188, 2021.
- [GKS21] Denisa O. C. Greconici, Matthias J. Kannwischer, and Amber Sprenkels. Compact Dilithium implementations on Cortex-M3 and Cortex-M4. *IACR TCHES*, 2021(1):1–24, 2021.
- [GM10] Shuhong Gao and Todd Mateer. Additive fast fourier transforms over finite fields. *IEEE Transactions on Information Theory*, 56(12):6265–6272, 2010.
- [HAZ⁺24] Junhao Huang, Alexandre Adomnicai, Jipeng Zhang, Wangchen Dai, Yao Liu, Ray C. C. Cheung, Çetin Kaya Koç, and Donglong Chen. Revisiting Keccak and Dilithium implementations on ARMv7-M. *IACR TCHES*, 2024(2):1–24, 2024.

- [HZZ⁺22] Junhao Huang, Jipeng Zhang, Haosong Zhao, Zhe Liu, Ray C. C. Cheung, Çetin Kaya Koç, and Donglong Chen. Improved plantard arithmetic for lattice-based cryptography. *IACR TCHES*, 2022(4):614–636, 2022.
- [JLK⁺24] Jihoon Jang, Myeonghoon Lee, Suhri Kim, Seogchung Seo, and Seokhie Hong. Optimized implementation of $\text{GF}(2)[x]$ multiplication for HQC on AVX2. *Journal of the Korea Institute of Information Security & Cryptology*, 34(5):841–853, 2024.
- [KPR⁺] Matthias J. Kannwischer, Richard Petri, Joost Rijneveld, Peter Schwabe, and Ko Stoffelen. PQM4: Post-quantum crypto library for the ARM Cortex-M4. <https://github.com/mupq/pqm4>.
- [LANH16] Sian-Jheng Lin, Tareq Y Al-Naffouri, and Yung-Hsiang S Han. FFT algorithm for binary extension finite fields and its application to reed-solomon codes. *IEEE Transactions on Information Theory*, 62(10):5343–5358, 2016.
- [LCH14] Sian-Jheng Lin, Wei-Ho Chung, and Yung-Hsiang S. Han. Novel polynomial basis and its application to reed-solomon erasure codes. In *55th FOCS*, pages 316–325. IEEE Computer Society Press, October 2014.
- [LCK⁺18] Wen-Ding Li, Ming-Shing Chen, Po-Chun Kuo, Chen-Mou Cheng, and Bo-Yin Yang. Frobenius additive fast fourier transform. In *Proceedings of the 2018 ACM International Symposium on Symbolic and Algebraic Computation*, pages 263–270, 2018.
- [LJKH25] Myeonghoon Lee, Jihoon Jang, Suhri Kim, and Seokhie Hong. Improved frobenius fft for code-based cryptography on cortex-m4. *IEEE Internet of Things Journal*, 12(17):36318–36327, 2025.
- [MPC00] Lauren May, Lyta Penna, and Andrew J. Clark. An implementation of bitsliced DES on the Pentium MMXTM processor. In Ed Dawson, Andrew Clark, and Colin Boyd, editors, *ACISP 00*, volume 1841 of *LNCS*, pages 112–122. Springer, Berlin, Heidelberg, July 2000.
- [VzGG96] Joachim Von zur Gathen and Jürgen Gerhard. Arithmetic and factorization of polynomial over F_2 . In *Proceedings of the 1996 international symposium on Symbolic and algebraic computation*, pages 1–9, 1996.
- [XZL⁺20] Zejun Xiang, Xiangyong Zeng, Da Lin, Zhenzhen Bao, and Shasha Zhang. Optimizing implementations of linear layers. *IACR Trans. Symm. Cryptol.*, 2020(2):120–145, 2020.